Design and Kinematic Analysis of a SCARA Manipulator

NAME: NAMITHA . N. REDDY

COURSE:ROBOTICS PROJECT

ABSTRACT:

This project involves the design and simulation of a SCARA (Selective Compliance Assembly Robot Arm) manipulator used for pick-and-place tasks. The robot structure was designed using TinkerCAD , and its motion was modeled in MATLAB using forward and inverse kinematics. Inverse kinematics was applied to calculate the required joint angles for moving the end effector to desired positions. The model was tested through a linear simulation in MATLAB, where the pick-and-place operation was successfully verified. To visualize the movements , an animation was created in Blender using keyframes to show joint motion. Although object handling was not fully animated due to software limitations, the simulation results confirm the correct working of the manipulator.

INTRODUCTION:

- Robotic manipulators are widely used in industries for tasks such as pick-and-place, assembly, and material handling.
- The SCARA (Selective Compliance Assembly Robot Arm) manipulator is commonly used because of its simple design, speed, and accuracy.SCARA manipulators are mainly used for high-speed

pick-and-place, assembly, and material handling applications in industrial automation.

- A SCARA robot mainly consists of a fixed base, rotating arm links, and an end effector.
- In this project, a 3-DOF SCARA manipulator was designed to understand basic robotic motion.

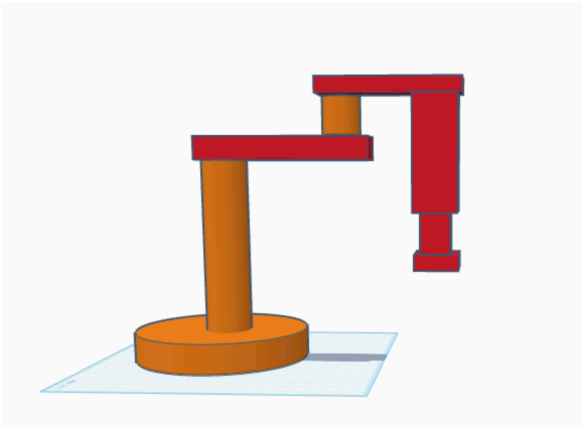
SYSTEM OVERVIEW:

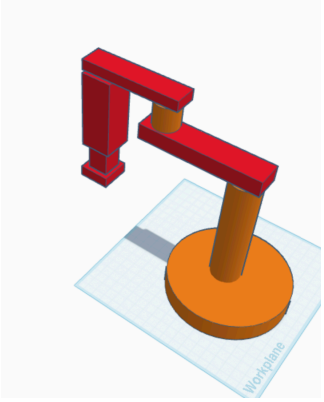
SCARA MANIPULATOR DESCRIPTION:

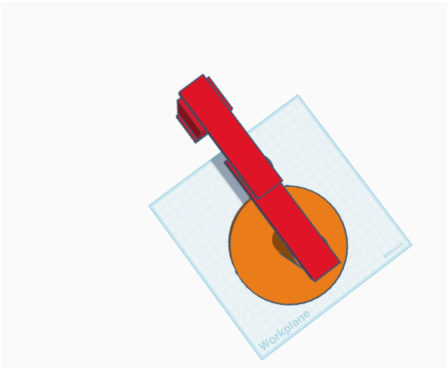
- The designed system is a 3-degree-of-freedom (3-DOF) SCARA manipulator intended for pick-and-place operations.
- The manipulator consists of a fixed base, two horizontal arm links, and an end effector arranged in a serial structure.
- **Degree of Freedom 1 (DOF 1):**
  The first joint is a revolute joint located at the
- base, which allows the entire arm to rotate about the vertical (Z) axis.
- **Degree of Freedom 2 (DOF 2):**
  The second joint is also a revolute joint, located between the first and second arm links, allowing the arm to extend and retract within the horizontal plane.
- **Degree of Freedom 3 (DOF 3):**
  The third degree of freedom provides vertical motion of the end effector, enabling pick-and-place action. This motion can be considered a prismatic joint or controlled vertical displacement.

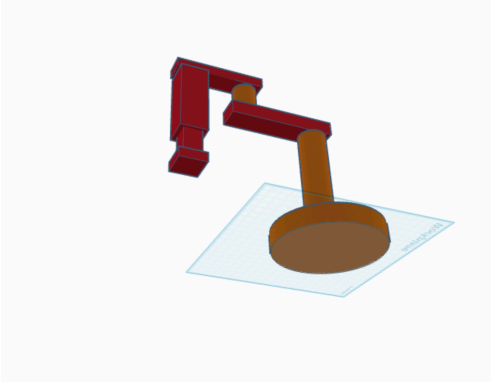
- The workspace of the manipulator is a circular annular region in the horizontal plane, defined by the lengths of the two arm links , with limited vertical movement for object handling.

  SCREENSHOTS OF THE TINKER CAD MODEL:









Software tools used:

- **TinkerCAD** – mechanical design
- **MATLAB** – kinematic modeling and IK
- **Blender** – motion animation and visualization

KINEMATIC MODELING:

Kinematic modeling was used to control the motion of the SCARA manipulator and generate a pick-and- place operation.

--->INVERSE KINEMATICS: sed to determine the joint angles of the SCARA manipulator based on a desired end effector position

a) Desired End-Effector Position

- The desired positions of the end effector were defined in Cartesian space as home, object (pick), and target (place) coordinates.
- These positions include X, Y, and Z values and represent the pick-and-place task.

b) For each point along the trajectory, the radial distance was computed and inverse kinematics equations were applied to calculate joint angles θ_1 and θ_2.

The vertical displacement was calculated separately to control the end-effector height.

−−>FORWARD KINEMATICS:used to calculate the positions of the joints and end effector using the joint angles obtained from inverse kinematics.

a) The joint angles θ_1 and θ_2 were used to compute the positions of the arm links in the X–Y plane.

b) The position of the end effector was calculated by combining the link transformations.

c) These calculated positions were used to plot and animate the SCARA manipulator in MATLAB.

Screenshots from the code; I have attached the code file in the zip file.

1. Definition of SCARA manipulator link lengths and dimensions in MATLAB.

```
%% ---------------- Robot Parameters (mm) ----------------
L1 = 120;
L2 = 100;

base_height     = 20;
column_height   = 120;
effector_height = 10;

z_home = base_height + column_height - effector_height/2;
z_pick = z_home - 50;
```

2. Desired Positions & Trajectory

```
%% ---------------- Key Cartesian Points ----------------
home   = [150   0   z_home];
object = [120  60   z_home];
target = [180 -60   z_home];
```

```
z = trajectory(3,1);

%% Inverse Kinematics
r = sqrt(x^2 + y^2);
theta2 = acos((r^2 - L1^2 - L2^2)/(2*L1*L2));
theta1 = atan2(y,x) - atan2(L2*sin(theta2), L1+L2*cos(theta2));
d3 = base_height + column_height - (z + effector_height/2);
```

3. Inverse Kinematics Code

Inverse kinematics equations used to compute joint angles.

Equations used:

$$r = \sqrt{x^2 + y^2}\theta_2 = \cos^{-1}\left(\frac{r^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

$$\theta_1 = \tan^{-1}\left(\frac{y}{x}\right) - \tan^{-1}\left(\frac{L_2\sin(\theta_2)}{L_1 + L_2\cos(\theta_2)}\right)$$

$$d_3 = h_{base} + h_{column} - \left(z + \frac{h_{effector}}{2}\right)$$

4. Forward Kinematics Code

Equations used:

$\mathbf{x_1 = L_1 \cos(\theta_1)}$

$\mathbf{y_1 = L_1 \sin(\theta_1)}$

$\mathbf{x_2 = x_1 + L_2 \cos(\theta_1 + \theta_2)}$

$\mathbf{y_2 = y_1 + L_2 \sin(\theta_1 + \theta_2)}$

```
%% Forward Kinematics
z0 = base_height + column_height;
x1 = L1*cos(theta1); y1 = L1*sin(theta1);
x2 = x1 + L2*cos(theta1+theta2);
y2 = y1 + L2*sin(theta1+theta2);
z3 = z0 - d3;
```

CONCLUSION:

- Successful inverse kinematics implementation
- Correct end-effector positioning
- Smooth joint motion in animation

- Validation through MATLAB plots and animation
- The project can be extended using ROS for better control and modular robot programming.
- Simulation realism can be improved by using Gazebo, which supports physics-based interaction and collision handling.
- Applying proper joint constraints and gripper mechanisms would improve object pickup accuracy.
- Adding basic sensors such as encoders or proximity sensors would enable feedback and closed-loop control.